Actividad Integradora 1: Herramienta propia de EDA automatizado (AutoEDA)

Estudiante: Josué Yarael Chiriboga Mite

Universidad Ecotec

Asignatura: Aplicaciones de Machine Learning

Docente: Carlos Andres Saavedra Arancibia

24 de octubre de 2026

Objetivo de la actividad

Programar una herramienta propia de análisis exploratorio de datos (EDA) automatizado en Python, capaz de procesar un dataset y generar de forma 100 % automatizada un diagnóstico estadístico, visual y de calidad de datos, optimizado para identificar los problemas clave de los modelos de regresión: la multicolinealidad, el sesgo y los valores atípicos. Este trabajo debe cubrir los diez puntos de EDA mas necesarios vistos en clase, implementarse como una clase reutilizable y exportarse con el comando mágico %%writefile.

Un requisito central de esta entrega es metodológico: las alertas estadísticas deben presentarse como observaciones que sustentan una decisión humana, la herramienta no toma nunca decisiones automáticas de limpieza, no elimina filas, no imputa valores ni transforma variables por su cuenta; se limita a diagnosticar y reportar.

Decisiones de diseño

Dataset.insurance.csv (Kaggle, Medical Cost Personal): 1338 registros y 7 variables con target continuo (charges), lo que lo hace adecuado para un problema de regresión.

Arquitectura. Clase reutilizable AutoEDA (programación orientada a objetos), exportada con el comando %%writefile al módulo herramientas_eda.py.

Referencia (benchmark).ydata-profiling y Sweetviz, herramientas consolidadas de EDA automático que permiten validar los resultados de la herramienta propia.

Principio metodológico. La herramienta solo diagnostica y reporta: cada acción (eliminar, imputar, transformar) se propone como recomendación para decisión humana en la sección final de interpretación.

Metodología: los 10 puntos de EDA

La clase AutoEDA implementa un método privado por cada punto de EDA y un orquestador run_analysis() que los ejecuta en orden con una sola llamada. Cada método imprime ta-

blas y gráficos y registra sus observaciones en una bitácora interna que se recupera con obtener_hallazgos().

Punto 1. Importar librerías.pandas, numpy, matplotlib y seaborn para manipulación y visualización; statsmodels para el cálculo del VIF.

Punto 2. Conocer el dataset.head(), tail(), shape, sample(), columns y memory_usage para dimensiones, memoria y muestras del dataset.

Punto 3. Tipo de variable.info(), dtypes y select_dtypes para clasificar columnas numéricas y categóricas.

Punto 4. Calidad de datos.duplicated() y búsqueda de valores centinela (?, -, N/A) y espacios sobrantes en las columnas de texto.

Punto 5. Valores faltantes.isnull().sum(), porcentaje de nulos por columna y clasificación de severidad (baja, media o alta).

Punto 6. Outliers.regla del IQR de Tukey con boxplots: cuartiles y límites de 1.5 × IQR y 3.0 × IQR (atípicos extremos).

Punto 7. Estadística descriptiva.describe(), media, mediana, desviación estándar, skewness y curtosis por variable numérica.

Punto 8. Desbalance de clases / target.en regresión se analiza la distribución del target continuo (media frente a mediana y skewness); en clasificación, value_counts(), porcentajes y countplot.

Punto 9. Análisis univariado.histogramas con KDE, skewness y curtosis por variable, con observación cuando |skew| > 1.0.

Punto 10. Análisis bivariado y multivariado.scatter con recta de tendencia, boxplots por categoría, heatmap de correlación, pares con $|r| \geq 0.80$ y VIF.

Resultados del diagnóstico

La ejecución automatizada sobre insurance.csv (target: charges) se completó en una sola llamada y registró cinco observaciones, cuyos valores fueron contrastados con los reportes de ydata-profiling y Sweetviz.

Calidad de datos y valores faltantes

El dataset tiene 1338 filas y 7 columnas, una fila duplicada exacta (0.07 %) y ningún va lor nulo; las columnas categóricas no contienen valores centinela ni espacios sobrantes.

Valores atípicos (regla del IQR)

bmi presenta 9 valores atípicos (0.67 %) y charges 139 (10.39 %), de los cuales 6 son extremos (3.0 × IQR). age y children no presentan valores atípicos.

Sesgo (skewness y curtosis)

charges tiene skewness de 1.52 y curtosis excesiva de 1.61, con media de 13 270 frente a mediana de 9 382; children roza el umbral con 0.94. age (0.06) y bmi (0.28) son prácticamente simétricas.

Multicolinealidad (correlación y VIF)

Ningún par de variables numéricas supera |r| = 0.80 y el VIF de age, bmi y children es 1.01, 1.01 y 1.00, por lo que no hay indicios de multicolinealidad.

Relaciones con el target

Las correlaciones de Pearson con charges son 0.299 (age), 0.198 (bmi) y 0.068 (children). El efecto dominante es categórico: el gasto medio es de 8 434 para no fumadores frente a 32 050 para fumadores; la región apenas diferencia (≈ 12 300–14 700).

Interpretación de los hallazgos y recomendaciones

Las observaciones emitidas por la herramienta son evidencia estadística, no órdenes de limpieza. A continuación se interpreta cada hallazgo con criterio de negocio y se proponen ac-

ciones a evaluar por el analista; ninguna transformación, eliminación o imputación se ejecuta automáticamente.

Calidad de datos

La fila duplicada es marginal (1 de 1338) y no altera de forma relevante ninguna estimación. Opción A: conservarla si se documenta que dos asegurados distintos pueden compartir todas las características. Opción B: eliminarla con df.drop_duplicates() si se concluye que es un error de captura. No se requiere estrategia de imputación porque no hay valores nulos.

Valores atípicos

Los valores altos de charges no parecen errores de captura: resultan de combinar smoker = yes con edad avanzada y bmi alto, es decir, gastos reales de la cola derecha de la población. Se recomienda no eliminarlos sin un análisis de negocio que lo justifique, evaluar una transformación logarítmica o un modelo robusto (HuberRegressor, RandomForestRegressor) y revisar los 9 valores atípicos de bmi (bmi > 47.3), que son clínicamente plausibles.

Sesgo del target

La asimetría de charges (skew = 1.52) es estructural: pocos asegurados generan gastos muy altos. Para modelos lineales viola el supuesto de residuos aproximadamente normales; se recomienda probar con y sin log(1 + charges) y comparar el RMSE en validación cruzada, adoptando la transformación solo si mejora la métrica. Box-Cox es una alternativa. Para children (0.94) no se recomienda actuar por ahora.

Multicolinealidad

Los predictores numéricos son linealmente independientes entre sí; no se recomienda eliminar ninguna variable por multicolinealidad. El VIF debe recalcularse sobre la matriz de diseño completa tras codificar las categóricas con one-hot.

Relaciones con el target

Se recomienda codificar las categóricas con one-hot (pd.get_dummies o OneHotEncoder) antes de modelar, priorizar smoker, age y bmi como predictores y conservar children y region salvo que una selección de características validada indique lo contrario.

Justificación de las decisiones

En esta sección explico, el porqué de cada decisión que tomé durante el desarrollo, apoyándome en los hallazgos verificados de la sección anterior.

Elección del dataset. Elegí insurance.csv porque necesitaba un problema de regresión con target continuo y charges, el gasto médico anual, cumple ese requisito. Además combina cuatro variables numéricas con tres categóricas, lo que me permitió ejercitar los diez puntos, y su tamaño compacto (1338 × 7) hace que el diagnóstico y los benchmark sean reproducibles en Colab. Al cargarlo desde una URL pública, el cuaderno funciona sin adjuntar archivos.

Arquitectura basada en clases. Implementé la solución como una clase y no como funciones sueltas porque el enunciado recomienda reutilizar el código y la programación orientada a objetos me permite encapsular el estado (el DataFrame y la bitácora) y ejecutar todo con una sola llamada a run_analysis(). En el constructor decidí trabajar sobre df.copy() porque quería garantizar que el diagnóstico jamás alterara el dataset original: una herramienta de EDA no debe ensuciar los datos que analiza.

Criterio de outliers: IQR y no Z-score. Elegí la regla del IQR de Tukey (1.5 × y 3.0 ×) porque se basa en cuartiles y es robusta frente a distribuciones sesgadas. Como charges tiene skewness de 1.52, un criterio que asume normalidad, como el Z-score, habría marcado aún más casos como atípicos sin que necesariamente lo fueran.

Umbrales explícitos. Definí umbrales conservadores y auditables ($|skew| > 1.0$, $|r| \geq 0.80$, VIF > 5 y VIF > 10) porque son valores de referencia habituales en la literatura y permiten que cada observación reporte su evidencia numérica junto al umbral.

Bitácora de observaciones. Decidí que cada alerta se registrara en una bitácora con punto, tipo y detalle porque así el diagnóstico queda trazable y puedo citarlo en este informe con evidencia numérica, en lugar de depender de mensajes sueltos en la consola.

No eliminar los outliers de charges. Decidí no eliminar los 139 valores atípicos de charges (10.39 % de las filas) porque al cruzarlos con la variable smoker se explica su origen: son gastos reales de fumadores, no errores de captura. Eliminarlos habría sesgado el modelo ha cia la población sana y habría sido una decisión automática, justo lo que el enunciado pide evi tar.

No transformar el target dentro de la herramienta. Decidí no aplicar log(1 + charges) en el diagnóstico a pesar del skewness de 1.52, porque la transformación es una decisión de modelado que debe validarse con métricas: propongo comparar el RMSE con y sin la transformación y adoptarla solo si mejora el resultado.

La fila duplicada. Decidí conservar la fila duplicada detectada (0.07 %) y dejarla como decisión documentada, porque su impacto es despreciable y no puedo descartar que corresponda a dos asegurados con características idénticas; la herramienta reporta la evidencia y deja la acción al analista.

Benchmark con herramientas consolidadas.I ncluí ydata-profiling y Sweetviz como referencia porque el enunciado las menciona y porque me permitieron validar que mis conteos de nulos, duplicados y distribuciones coinciden con los de herramientas consolidadas del ecosistema.

One-hot y VIF final. Para el modelado posterior recomiendo codificar las categóricas con one-hot y recalcular el VIF sobre la matriz de diseño completa, porque la multicolinealidad debe evaluarse con la representación final que verá el modelo, no solo con las tres variables numéricas originales cuyo VIF ≈ 1.0 no cubre a las dummies.

Referencias

American Psychological Association. (2020).Publication manual of the American Psychological Association(7.ª ed.). https://doi.org/10.1037/0000165-000

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment.Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kaggle. (s. f.).Medical cost personal datasets. https://www.kaggle.com/datasets/mirichoi0218/insurance

Lantz, B. (2013).Machine learning with R. Packt Publishing.

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt y J. Millman (Eds.),Proceedings of the 9th Python in Science Conference(pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Seabold, S., y Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. En S. van der Walt y J. Millman (Eds.),Proceedings of the 9th Python in Science Conference(pp. 92–96). https://doi.org/10.25080/Majora-92bf1922-011

Sweetviz. (s. f.).Sweetviz: An open source EDA package. https://pypi.org/project/sweetviz/

Waskom, M. L. (2021). Seaborn: Statistical data visualization.Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021

ydata-profiling. (s. f.).YData profiling. https://ydata-profiling.ydata.ai/

Anexo A

Código fuente de la clase AutoEDA

El código completo de la herramienta, con su guía técnica comentada por línea, se entrega en el módulo herramientas_eda.py. Se reproduce a continuación para consulta.

```
# -*- coding: utf-8 -*-
"""
herramientas_eda.py — Actividad Integradora 1 · Aplicaciones de Machine Lear-
ning
================================================================================
=
Herramienta PROPIA de Análisis Exploratorio de Datos (EDA) 100 % automatizada,
orientada a modelos de REGRESIÓN. Cubre los 10 puntos de EDA de clase y emite
un
diagnóstico estadístico, visual y de calidad de datos para detectar:

  · MULTICOLINEALIDAD  → matriz de correlación de Pearson + VIF (statsmodels).
  · SESGO (bias)       → skewness y curtosis de predictores y del target.
  · VALORES ATÍPICOS   → regla del rango intercuartílico (IQR) de Tukey + box-
plots.

⚠  PRINCIPIO METODOLÓGICO (lectura obligatoria antes de usar):
Esta clase SOLO DIAGNOSTICA Y REPORTA. Nunca elimina filas, imputa nulos ni
transforma variables de forma automática. Toda alerta estadística se presenta
como una OBSERVACIÓN (🔎) cuya acción de limpieza corresponde decidir al
analista, de forma documentada y con criterio de negocio.
"""

# ================== PASO 1. IMPORTAR LIBRERÍAS ==================
import math                                                    # módulo están-
dar: provee math.ceil() para calcular filas de subgráficos
import pandas as pd                                            # pandas: es-
tructuras tabulares (DataFrame/Series) y operaciones vectorizadas
import numpy as np                                             # numpy: tipos
numéricos base y funciones de álgebra/matemáticas
import matplotlib.pyplot as plt                                # matplotlib:
motor de renderizado de figuras (se usa el submódulo pyplot)
import seaborn as sns                                          # seaborn: grá-
ficos estadísticos de alto nivel (histplot, boxplot, heatmap...)

try:                                                           # bloque tole-
rante: si statsmodels no está instalado, el resto de la clase sigue operativo
    from statsmodels.stats.outliers_influence import variance_inflation_factor
# statsmodels: función VIF para diagnosticar multicolinealidad
except ImportError:                                            # se captura la
ausencia del paquete para degradar con elegancia (sin romper el EDA)
    variance_inflation_factor = None                           # marcador
None: los métodos de VIF avisarán que el paquete no está disponible

# Constantes de los criterios estadísticos usados como umbrales de OBSERVACIÓN
UMBRAL_SKEW = 1.0      # |skewness| > 1.0  → distribución asimétrica que con-
viene evaluar (no transformar automáticamente)
```

```
UMBRAL_CORR = 0.80     # |r de Pearson| >= 0.80 → par de predictores con correlación lineal alta
UMBRAL_VIF_MOD = 5.0   # VIF > 5  → indicio moderado de multicolinealidad
UMBRAL_VIF_ALTO = 10.0 # VIF > 10 → indicio severo de multicolinealidad
MULTIPLO_IQR = 1.5     # factor de Tukey: Q1 - 1.5·IQR y Q3 + 1.5·IQR delimitan los valores "atípicos leves"
MULTIPLO_IQR_EXTREMO = 3.0  # factor 3.0·IQR delimita los valores "atípicos extremos"


def _mostrar(objeto):
    """Muestra un DataFrame/objeto con la representación enriquecida si está disponible (notebook) o texto plano (script)."""
    try:                                                    # intenta primero el camino interactivo de Jupyter/Colab
        from IPython.display import display                 # display() renderiza tablas HTML interactivas dentro del notebook
        display(objeto)                                     # despliega el objeto usando su representación gráfica nativa
    except ImportError:                                     # fuera de un notebook no existe IPython.display
        print(objeto)                                       # respaldo textual: print() funciona en cualquier entorno Python


class AutoEDA:
    """Clase reutilizable que ejecuta los 10 puntos de EDA de clase sobre un DataFrame, sin modificar los datos originales."""

    def __init__(self, df, target_col=None, figsize=(10, 6)):
        """Constructor: recibe el DataFrame y (opcionalmente) el nombre de la variable objetivo.

        Parámetros
        ----------
        df : pd.DataFrame
            Dataset a diagnosticar. Se trabaja sobre una COPIA para no mutar los datos del usuario.
        target_col : str, opcional
            Nombre de la variable dependiente (target). Si es numérica se asume un problema de REGRESIÓN.
        figsize : tuple, opcional
            Tamaño (ancho, alto) en pulgadas de las figuras de matplotlib.
        """
        self.df = df.copy()                                 # copia defensiva: ningún método de la clase altera el DataFrame original
        self.target_col = target_col                        # guarda el nombre del target para los análisis bivariados (puntos 8 y 10)
        self.figsize = figsize                              # guarda el tamaño por defecto de las figuras generadas
        self.num_cols = self.df.select_dtypes(include=[np.number]).columns.tolist()   # columnas numéricas: enteros y flotantes (incluye booleanos 0/1)
        self.cat_cols = self.df.select_dtypes(include=['object', 'string', 'category']).columns.tolist()  # columnas categóricas/texto
        self.hallazgos = []                                 # bitácora: aquí se registran todas las OBSERVACIONES para la interpretación final

    # ------------------------------------------------------------------
    def _registrar(self, punto, tipo, detalle):
        """Registra una observación en la bitácora interna y la imprime con formato consistente.

        Parámetros
        ----------
        punto : int
            Número del punto de EDA (2 a 10) al que pertenece la observación.
```

```
        tipo : str
            Etiqueta corta de la observación (ej.: 'outliers', 'sesgo', 'multi-
colinealidad').
        detalle : str
            Texto explicativo con la evidencia numérica y la acción a EVALUAR
(no a ejecutar).
        """
        self.hallazgos.append({'punto': punto, 'tipo': tipo, 'detalle': deta-
lle})  # almacena la observación para el resumen final
        print(f"🔎 OBSERVACIÓN [P{punto} · {tipo}]: {detalle}")
# imprime la observación: es un aviso, NUNCA una acción automática

    # ================== PASO 2. CONOCER EL DATASET ==================
    def _paso2_conocer_dataset(self):
        """Punto 2 — Conocer el dataset: dimensiones, memoria, catálogo de co-
lumnas y muestras (head/tail/sample)."""
        print("\n=== PASO 2. CONOCER EL DATASET ===")           # encabezado
del bloque para lectura ágil del reporte
        print(f"Dimensiones: {self.df.shape[0]} filas × {self.df.shape[1]} co-
lumnas")   # .shape devuelve (n_filas, n_columnas)
        print(f"Memoria RAM estimada: {self.df.memory_usage(deep=True).sum() /
1024:.2f} KB")  # deep=True mide también los objetos internos de las cadenas
        print(f"Catálogo de columnas ({len(self.df.columns)}): {self.df.column-
s.tolist()}")     # .columns lista los nombres de todas las variables
        print("--- Primeras 3 filas (head) ---")                # head() ins-
pecciona el inicio del dataset
        _mostrar(self.df.head(3))                                # muestra las 3
primeras filas para validar formato y semántica
        print("--- Últimas 3 filas (tail) ---")                 # tail() ins-
pecciona el final (detecta registros truncados o mal cargados)
        _mostrar(self.df.tail(3))                                # muestra las 3
últimas filas
        print("--- Muestra aleatoria (sample) ---")             # sample() ins-
pecciona filas al azar, útiles en datasets ordenados por fecha
        _mostrar(self.df.sample(3, random_state=42))             #
random_state=42 fija la semilla para que la muestra sea reproducible

    # ================== PASO 3. TIPO DE VARIABLE ==================
    def _paso3_tipos_variable(self):
        """Punto 3 — Tipo de variable: clasifica cada columna (numérica vs. ca-
tegórica) mediante dtypes/select_dtypes."""
        print("\n=== PASO 3. TIPO DE VARIABLE ===")             # encabezado
del bloque
        tabla_tipos = pd.DataFrame({                            # construye una
tabla resumen de tipos
            'tipo': self.df.dtypes.astype(str),                  # .dtypes de-
vuelve el tipo pandas de cada columna (int64, float64, object...)
            'valores_unicos': self.df.nunique(),                 # .nunique()
cuenta valores distintos: útil para detectar IDs de alta cardinalidad
            'faltantes': self.df.isnull().sum(),                 # conteo rápido
de nulos como vista previa del punto 5
        })
        _mostrar(tabla_tipos)                                    # muestra la
tabla de tipos
        print(f"Numéricas ({len(self.num_cols)}): {self.num_cols}")   #
num_cols se calculó con select_dtypes(include=[np.number])
        print(f"Categóricas ({len(self.cat_cols)}): {self.cat_cols}") #
cat_cols se calculó con select_dtypes(include=['object','string','category'])
        for c in self.cat_cols:                                  # recorre las
categóricas buscando identificadores
            if self.df[c].nunique() == len(self.df):             # si hay tantos
valores únicos como filas, la columna identifica cada registro
                self._registrar(3, 'identificador',              # registra la
observación en la bitácora
                                f"'{c}' tiene {self.df[c].nunique()} valores
únicos (uno por fila): "
```

```
                                f"evaluar si es un ID sin valor predictivo.")

    # ================== PASO 4. CALIDAD DE DATOS ==================
    def _paso4_calidad_datos(self):
        """Punto 4 — Calidad de datos: duplicados exactos y valores centinela
('?', '-', 'N/A', espacios sobrantes)."""
        print("\n=== PASO 4. CALIDAD DE DATOS ===")             # encabezado
del bloque
        n_dup = self.df.duplicated().sum()                      # .duplicated()
marca las filas idénticas a una anterior; .sum() las cuenta
        print(f"Filas duplicadas exactas: {n_dup} ({100 * n_dup /
len(self.df):.2f}%)")  # porcentaje sobre el total de filas
        if n_dup > 0:                                           # solo emite
observación si realmente existen duplicados
            self._registrar(4, 'duplicados',                    # registra la
observación (no los elimina: decisión humana)
                            f"{n_dup} filas duplicadas detectadas: evaluar si
se eliminan con "
                            f"df.drop_duplicates() o si corresponden a regis-
tros legítimos repetidos.")
        centinelas = ['?', '-', 'n/a', 'na', 'none', 'null', '']  # lista de
valores típicos que "disfrazan" datos ausentes
        for c in self.cat_cols:                                 # la búsqueda
de centinelas se aplica a columnas de texto
            normalizada = self.df[c].astype(str).str.strip().str.lower()  #
convierte a texto, quita espacios y pasa a minúsculas para comparar
            n_raros = normalizada.isin(centinelas).sum()        # cuenta coin-
cidencias exactas con los valores centinela
            n_espacios = (self.df[c].astype(str) !=
self.df[c].astype(str).str.strip()).sum()  # detecta textos con espacios al
inicio o final
            if n_raros > 0 or n_espacios > 0:                   # si hay ha-
llazgos en esta columna...
                self._registrar(4, 'valores_raros',             # ...se regis-
tra la observación con el detalle numérico
                                f"'{c}': {n_raros} valor(es) centinela ('?',
'-', 'N/A'...) y "
                                f"{n_espacios} texto(s) con espacios sobrantes:
evaluar normalización manual.")
            else:                                               # si la columna
está limpia...
                print(f"✓ '{c}': sin valores centinela ni espacios sobrantes.")
# ...se reporta explícitamente como verificada

    # ================== PASO 5. VALORES FALTANTES ==================
    def _paso5_valores_faltantes(self):
        """Punto 5 — Valores faltantes: conteo, porcentaje y clasificación de
severidad (sin imputar nada)."""
        print("\n=== PASO 5. VALORES FALTANTES ===")             # encabezado
del bloque
        nulos = self.df.isnull().sum()                          # .isnull()
marca cada celda vacía; .sum() cuenta los nulos por columna
        pct = (nulos / len(self.df)) * 100                      # convierte el
conteo a porcentaje sobre el total de filas
        tabla_nulos = pd.DataFrame({'nulos': nulos, 'porcentaje(%)':
pct.round(2)})  # tabla resumen ordenable por severidad
        _mostrar(tabla_nulos)                                   # muestra la
tabla completa de nulos
        for c in self.df.columns:                               # recorre todas
las columnas del dataset
            if nulos[c] > 0:                                    # solo clasifi-
ca las columnas que tienen nulos
                if pct[c] < 5:                                  # umbral 1: me-
nos del 5 % de nulos
                    severidad = 'baja → evaluar imputación simple (media/media-
na/moda)'
```

```
                elif pct[c] <= 30:                                  # umbral 2: en-
tre 5 % y 30 % de nulos
                    severidad = 'media → evaluar imputación avanzada o modelado
específico'
                else:                                               # más del 30 %
de nulos
                    severidad = 'alta → evaluar descarte de la columna o trata-
miento especial'
                self._registrar(5, 'nulos',                         # registra la
observación indicando severidad y opciones a EVALUAR
                                f"'{c}': {nulos[c]} nulos ({pct[c]:.2f}%) → se-
veridad {severidad}. "
                                f"La herramienta NO imputa: la estrategia la
define el analista.")
        if nulos.sum() == 0:                                        # caso más fre-
cuente en datasets limpios
            print("✓ No se detectaron valores nulos (NaN/None): no se requiere
estrategia de imputación.")  # confirma la ausencia de nulos

    # ================== PASO 6. OUTLIERS ==================
    def _paso6_outliers(self):
        """Punto 6 — Outliers: regla del IQR de Tukey (1.5× y 3.0×) sobre cada
variable numérica + boxplots."""
        print("\n=== PASO 6. OUTLIERS (REGLA DEL IQR) ===")     # encabezado
del bloque
        filas = []                                                  # acumulador de
resúmenes por variable
        for c in self.num_cols:                                     # la regla del
IQR solo aplica a variables numéricas
            q1 = self.df[c].quantile(0.25)                          # primer cuar-
til Q1: valor bajo el cual queda el 25 % de los datos
            q3 = self.df[c].quantile(0.75)                          # tercer cuar-
til Q3: valor bajo el cual queda el 75 % de los datos
            iqr = q3 - q1                                           # rango inter-
cuartílico: dispersión del 50 % central
            lim_inf = q1 - MULTIPLO_IQR * iqr                       # límite infe-
rior de Tukey (1.5 × IQR)
            lim_sup = q3 + MULTIPLO_IQR * iqr                       # límite supe-
rior de Tukey (1.5 × IQR)
            n_out = ((self.df[c] < lim_inf) | (self.df[c] > lim_sup)).sum()   #
cuenta valores fuera del intervalo [lim_inf, lim_sup]
            n_ext = ((self.df[c] < q1 - MULTIPLO_IQR_EXTREMO * iqr) |       #
umbral 3.0 × IQR para atípicos extremos
                     (self.df[c] > q3 + MULTIPLO_IQR_EXTREMO * iqr)).sum()
            filas.append({'variable': c, 'Q1': round(q1, 2), 'Q3': round(q3,
2), 'IQR': round(iqr, 2),
                          'atipicos': n_out, 'atipicos_%': round(100 * n_out /
len(self.df), 2),
                          'extremos': n_ext})                    # agrega el
resumen de la variable a la lista
            if n_out > 0:                                           # emite obser-
vación solo si la variable tiene atípicos
                self._registrar(6, 'outliers',                      # registra la
observación: sugiere VERIFICAR, nunca borrar
                                f"'{c}': {n_out} valores atípicos ({100 * n_out
/ len(self.df):.2f}%) y {n_ext} extremos. "
                                f"Verificar si son anomalías reales o errores
de captura antes de decidir su tratamiento.")
        _mostrar(pd.DataFrame(filas))                               # muestra la
tabla resumen de outliers
        ncols = 3                                                   # número de bo-
xplots por fila en la cuadrícula
        nrows = math.ceil(len(self.num_cols) / ncols)               # calcula cuán-
tas filas de subgráficos se necesitan
        fig, axes = plt.subplots(nrows, ncols, figsize=(4 * ncols, 3 * nrows))
# crea la cuadrícula de ejes
```

```
        axes = axes.flatten()                                       # convierte la
matriz de ejes en lista plana para iterar fácil
        for i, c in enumerate(self.num_cols):                       # recorre cada
variable numérica con su subgráfico
            sns.boxplot(data=self.df, x=c, ax=axes[i])              # boxplot hori-
zontal: mediana, caja Q1–Q3, bigotes y puntos atípicos
            axes[i].set_title(f'Boxplot de {c}')                    # titula el su-
bgráfico con el nombre de la variable
        for j in range(len(self.num_cols), len(axes)):              # recorre los
ejes sobrantes (si sobran celdas en la cuadrícula)
            axes[j].axis('off')                                     # apaga los
ejes vacíos para una figura limpia
        plt.tight_layout()                                          # ajusta márge-
nes para que títulos y etiquetas no se solapen
        plt.show()                                                  # renderiza la
figura (en notebook: salida en línea; en script: ventana/archivo)

    # ================== PASO 7. ESTADÍSTICA DESCRIPTIVA ==================
    def _paso7_estadistica_descriptiva(self):
        """Punto 7 — Estadística descriptiva: tendencia central, dispersión,
asimetría y curtosis de las numéricas."""
        print("\n=== PASO 7. ESTADÍSTICA DESCRIPTIVA ===")      # encabezado
del bloque
        if not self.num_cols:                                       # si no hay va-
riables numéricas no hay nada que describir
            print("⚠ No hay variables numéricas en el dataset.")  # aviso ex-
plícito en lugar de fallar
            return                                                  # termina el
método sin error
        desc = self.df[self.num_cols].describe().T                  # describe()
calcula count, mean, std, min, cuartiles y max; .T transpone
        desc['rango'] = desc['max'] - desc['min']                   # amplitud to-
tal: distancia entre el máximo y el mínimo
        desc['cv'] = desc.apply(                                    # coeficiente
de variación CV = std/media (dispersión relativa)
            lambda fila: (fila['std'] / fila['mean']) if fila['mean'] != 0 else
np.nan,  # evita dividir entre cero cuando media=0
            axis=1).round(2)                                        # axis=1 aplica
la función fila por fila; .round(2) limita decimales
        desc['skewness'] = self.df[self.num_cols].skew().round(2)   # skew():
asimetría muestral (>0 cola derecha, <0 cola izquierda)
        desc['curtosis'] = self.df[self.num_cols].kurt().round(2)   # kurt():
curtosis EXCESIVA (0 = normal; >0 colas pesadas)
        _mostrar(desc[['mean', 'std', 'min', '50%', 'max', 'rango', 'cv',
'skewness', 'curtosis']])  # muestra las columnas relevantes
        print("· '50%' es la mediana (percentil 50): si difiere mucho de la me-
dia, la distribución está sesgada.")  # ayuda interpretativa

    # ================== PASO 8. DESBALANCE / TARGET ==================
    def _paso8_target(self):
        """Punto 8 — Variable objetivo: distribución (regresión, target conti-
nuo) o balance de clases (clasificación)."""
        print("\n=== PASO 8. VARIABLE OBJETIVO (TARGET) ===")   # encabezado
del bloque
        if self.target_col is None:                                 # la clase tam-
bién debe funcionar sin target (análisis exploratorio puro)
            print("ℹ️ No se declaró target_col: se omite el punto 8 (pasar tar-
get_col='nombre' al instanciar).")
            return                                                  # termina el
método sin error
        if self.target_col not in self.df.columns:                  # valida que el
nombre del target exista realmente en el DataFrame
            print(f"⚠ El target '{self.target_col}' no existe entre las colum-
nas: {self.df.columns.tolist()}")
            return                                                  # termina el
método sin error
```

```
        if self.target_col in self.num_cols:                    # RAMA REGRE-
SIÓN: el target es continuo
            s = self.df[self.target_col]                        # Serie con los
valores del target
            print(f"Target continuo '{self.target_col}': media={s.mean():.2f} ·
mediana={s.median():.2f} · "
                  f"min={s.min():.2f} · max={s.max():.2f} ·
skew={s.skew():.2f}")  # resumen de forma y ubicación
            if abs(s.skew()) > UMBRAL_SKEW:                     # compara la
asimetría con el umbral de observación
                self._registrar(8, 'sesgo_target',              # registra la
observación sobre el target
                                f"El target '{self.target_col}' tiene |skew|
={abs(s.skew()):.2f} > {UMBRAL_SKEW}: "
                                f"la media se aleja de la mediana. Evaluar
transformación log(1+x) o Box-Cox "
                                f"SOLO si el modelo lo requiere (decisión huma-
na, no automática).")
            plt.figure(figsize=self.figsize)                    # crea una fi-
gura nueva con el tamaño por defecto
            sns.histplot(s, kde=True, bins=30)                  # histograma
con curva de densidad KDE: revela la forma de la distribución
            plt.axvline(s.mean(), color='red', linestyle='--',
label=f'media={s.mean():.0f}')    # línea vertical en la media
            plt.axvline(s.median(), color='green', linestyle='--', label=f'me-
diana={s.median():.0f}')  # línea vertical en la mediana
            plt.title(f'Distribución del target: {self.target_col}')  # título
descriptivo de la figura
            plt.legend()                                        # muestra la
leyenda con media y mediana
            plt.show()                                          # renderiza la
figura
        else:                                                   # RAMA CLASIFI-
CACIÓN: el target es categórico
            conteos = self.df[self.target_col].value_counts(normalize=True) *
100  # value_counts(normalize=True) devuelve proporciones por clase
            print(f"Distribución de clases del target '{self.target_col}'
(%):")  # encabezado de la tabla de balance
            _mostrar(conteos.round(2))                          # muestra los
porcentajes de cada clase
            if conteos.min() < 20:                              # regla prácti-
ca: clase minoritaria por debajo del 20 % = desbalance
                self._registrar(8, 'desbalance',                # registra la
observación de desbalance
                                f"La clase minoritaria '{conteos.idxmin()}' re-
presenta {conteos.min():.1f}%: "
                                f"evaluar técnicas de balanceo (SMOTE,
class_weight) según el modelo elegido.")
            plt.figure(figsize=self.figsize)                    # crea la figu-
ra
            sns.countplot(data=self.df, x=self.target_col)      # countplot:
barras con la frecuencia de cada clase
            plt.title(f'Balance de clases: {self.target_col}')  # título de la
figura
            plt.show()                                          # renderiza la
figura

    # ================== PASO 9. ANÁLISIS UNIVARIADO ==================
    def _paso9_analisis_univariado(self):
        """Punto 9 — Análisis univariado: histogramas + KDE y diagnóstico de
sesgo (skewness) y curtosis por variable."""
        print("\n=== PASO 9. ANÁLISIS UNIVARIADO (DISTRIBUCIONES) ===")  # en-
cabezado del bloque
        if not self.num_cols:                                   # guarda: sin
numéricas no aplica
            print("⚠ No hay variables numéricas en el dataset.")
```

```
            return                                                  # termina el
método sin error
        tabla_forma = pd.DataFrame({                               # tabla compara-
tiva de forma de cada distribución
            'skewness': self.df[self.num_cols].skew().round(2),     # asimetría
por variable
            'curtosis': self.df[self.num_cols].kurt().round(2),     # curtosis
excesiva por variable
        })
        _mostrar(tabla_forma)                                       # muestra la
tabla de forma
        for c in self.num_cols:                                     # recorre cada
variable numérica
            if abs(tabla_forma.loc[c, 'skewness']) > UMBRAL_SKEW:  # aplica el
umbral de sesgo severo
                self._registrar(9, 'sesgo',                         # registra la
observación de asimetría
                                f"'{c}': |skew|={abs(tabla_forma.loc[c, 'skew-
ness']):.2f} > {UMBRAL_SKEW} → "
                                f"distribución asimétrica. Evaluar transforma-
ción (log(1+x), Box-Cox) según el modelo: "
                                f"no es una acción automática.")
        ncols = 3                                                   # número de
histogramas por fila
        nrows = math.ceil(len(self.num_cols) / ncols)           # filas necesa-
rias de la cuadrícula
        fig, axes = plt.subplots(nrows, ncols, figsize=(4 * ncols, 3 * nrows))
# crea la cuadrícula de ejes
        axes = axes.flatten()                                       # aplana los
ejes para iterar
        for i, c in enumerate(self.num_cols):                       # recorre cada
variable con su subgráfico
            sns.histplot(self.df[c], kde=True, bins=30, ax=axes[i], color='sky-
blue')  # histograma + curva KDE de la variable
            axes[i].set_title(f'Distribución de {c} (skew={tabla_forma.loc[c,
"skewness"]})')  # título con la asimetría
        for j in range(len(self.num_cols), len(axes)):          # apaga ejes
sobrantes
            axes[j].axis('off')                                     # oculta celdas
vacías de la cuadrícula
        plt.tight_layout()                                          # ajusta márge-
nes
        plt.show()                                                  # renderiza la
figura

    # ================== PASO 10. BIVARIADO / MULTIVARIADO ==================
    def _paso10_bivariado_multivariado(self):
        """Punto 10 — Bivariado/multivariado: relación con el target, correla-
ción (heatmap) y multicolinealidad (VIF)."""
        print("\n=== PASO 10. ANÁLISIS BIVARIADO Y MULTIVARIADO ===")  # enca-
bezado del bloque
        # --- 10a. Predictores NUMÉRICOS vs target: scatter con recta de ten-
dencia ---
        if self.target_col and self.target_col in self.num_cols:  # solo aplica
si hay target numérico declarado
            corr_target = self.df[self.num_cols].corr()
[self.target_col].drop(self.target_col)  # correlación Pearson de cada numérica
con el target
            corr_target = corr_target.sort_values(key=abs, ascending=False)  #
ordena por valor absoluto: las más influyentes primero
            print(f"Correlación de Pearson de cada predictor numérico con '{se-
lf.target_col}':")  # encabezado del ranking
            _mostrar(corr_target.round(3))                      # muestra el
ranking de correlaciones con el target
            for c in [x for x in self.num_cols if x != self.target_col]:  # re-
corre los predictores numéricos (excluye el target)
```

```
                plt.figure(figsize=self.figsize)                # figura por
cada relación predictor–target
                sns.regplot(data=self.df, x=c, y=self.target_col,  # regplot:
nube de puntos + recta de regresión lineal
                            scatter_kws={'alpha': 0.35},        # transparencia
de los puntos para ver densidad
                            line_kws={'color': 'red'})          # recta de ten-
dencia en rojo
                plt.title(f'{c} vs {self.target_col} (r={corr_target[c]:.3f})')
# título con el coeficiente de Pearson
                plt.show()                                      # renderiza la
figura
        # --- 10b. Predictores CATEGÓRICOS vs target: boxplots agrupados ---
        if self.target_col and self.target_col in self.df.columns:  # aplica a
cualquier target declarado (numérico o categórico)
            for c in self.cat_cols:                             # recorre cada
variable categórica
                plt.figure(figsize=self.figsize)                # figura por
categoría
                sns.boxplot(data=self.df, x=c, y=self.target_col)  # boxplot
del target por cada categoría: revela efectos de grupo
                plt.title(f'{self.target_col} por {c}')         # título des-
criptivo
                plt.show()                                      # renderiza la
figura
        # --- 10c. Matriz de correlación (heatmap) y pares colineales ---
        if len(self.num_cols) > 1:                              # se necesitan
al menos 2 numéricas para correlacionar
            corr = self.df[self.num_cols].corr()                # matriz de co-
rrelación lineal de Pearson entre todas las numéricas
            plt.figure(figsize=(8, 6))                          # figura cua-
drada para el heatmap
            sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',  # heat-
map anotado: color frío para negativos, cálido para positivos
                        center=0, linewidths=0.5)               # center=0 an-
cla la escala de color en correlación nula
            plt.title('Matriz de correlación de Pearson (variables numéricas)')
# título del heatmap
            plt.tight_layout()                                  # ajusta márge-
nes
            plt.show()                                          # renderiza el
heatmap
            for i in range(len(self.num_cols)):                 # doble bucle
para revisar TODOS los pares sin repetir
                for j in range(i + 1, len(self.num_cols)):      # j inicia en
i+1: solo la mitad triangular superior
                    v = corr.iloc[i, j]                         # coeficiente
del par (i, j)
                    if abs(v) >= UMBRAL_CORR:                   # aplica el um-
bral de alta correlación
                        self._registrar(10, 'multicolinealidad',  # registra el
par colineal como observación
                                        f"'{self.num_cols[i]}' y
'{self.num_cols[j]}' con |r|={abs(v):.2f} ≥ {UMBRAL_CORR}: "
                                        f"evaluar eliminar una de las dos o
combinarlas antes de un modelo lineal.")
            if not any(abs(corr.iloc[i, j]) >= UMBRAL_CORR      # verifica si
NO hubo ningún par por encima del umbral
                       for i in range(len(self.num_cols)) for j in range(i + 1,
len(self.num_cols))):
                print(f"✓ Ningún par de variables numéricas supera |r| = {UM-
BRAL_CORR}.")  # confirma ausencia de correlación extrema
        # --- 10d. VIF: Factor de Inflación de la Varianza ---
        if variance_inflation_factor is None:                   # si statsmo-
dels no se instaló en el entorno...
```

```
            print("⚠ VIF no disponible: instala statsmodels (pip install stats-
models) para el diagnóstico de multicolinealidad.")
            return                                                  # termina solo
este bloque, no el resto del análisis
        predictores = [c for c in self.num_cols if c != self.target_col]  # ex-
cluye el target del cálculo de VIF
        if len(predictores) > 1:                                    # el VIF exige
al menos 2 predictores
            X = self.df[predictores].dropna()                       # VIF requiere
matriz sin nulos ni constantes; dropna garantiza lo primero
            try:                                                    # protege
contra columnas constantes (varianza cero) que invalidan el VIF
                tabla_vif = pd.DataFrame({                          # construye la
tabla de resultados
                    'variable': predictores,                        # nombre de
cada predictor
                    'VIF': [round(variance_inflation_factor(X.values, i), 2)
for i in range(len(predictores))],  # VIF_i = 1/(1-R²_i) de cada predictor
frente a los demás
                })
                _mostrar(tabla_vif)                                 # muestra la
tabla VIF
                for i, c in enumerate(predictores):                 # recorre los
resultados para clasificar la severidad
                    if tabla_vif.loc[i, 'VIF'] > UMBRAL_VIF_ALTO:      # VIF >
10: multicolinealidad severa
                        self._registrar(10, 'vif',                  # registra la
observación severa
                                        f"'{c}': VIF={tabla_vif.loc[i,
'VIF']:.2f} > {UMBRAL_VIF_ALTO} → multicolinealidad severa: "
                                        f"evaluar eliminar la variable o combi-
narla con otra.")
                    elif tabla_vif.loc[i, 'VIF'] > UMBRAL_VIF_MOD:     # VIF
entre 5 y 10: multicolinealidad moderada
                        self._registrar(10, 'vif',                  # registra la
observación moderada
                                        f"'{c}': VIF={tabla_vif.loc[i,
'VIF']:.2f} > {UMBRAL_VIF_MOD} → multicolinealidad moderada: "
                                        f"monitorear su efecto sobre los coefi-
cientes del modelo.")
                if (tabla_vif['VIF'] <= UMBRAL_VIF_MOD).all():  # si todos los
VIF quedan por debajo del umbral moderado
                    print(f"✓ VIF de todos los predictores ≤ {UMBRAL_VIF_MOD}:
sin indicios de multicolinealidad.")  # confirma diagnóstico sano
            except ValueError:                                      # captura el
caso de columnas constantes u otros fallos numéricos
                print("⚠ No se pudo calcular el VIF (posible columna constante
o singularidad numérica).")  # aviso sin romper el flujo

    # ================== ORQUESTADOR PRINCIPAL ==================
    def run_analysis(self):
        """Método público: ejecuta los 10 puntos de EDA en orden y de forma 100
% automatizada (una sola llamada)."""
        print("=" * 72)                                             # línea separa-
dora superior del reporte
        print(" DIAGNÓSTICO AUTOMATIZADO DE EDA · ORIENTADO A REGRESIÓN ".cen-
ter(72))  # título centrado del reporte
        print("=" * 72)                                             # línea separa-
dora
        print("⚠  Esta herramienta SOLO diagnostica y reporta: no elimina fi-
las, no imputa")
        print("   nulos y no transforma variables. Cada acción de limpieza es
una decisión")
        print("   HUMANA que debe documentarse en la sección de interpreta-
ción.\n")
```

```
        print("PASO 1 (importar librerías) se cumple en la cabecera de este mó-
dulo:")  # aclaración del punto 1
        print("pandas, numpy, matplotlib, seaborn y statsmodels (para el VIF).\
n")
        self._paso2_conocer_dataset()                           # ejecuta el
punto 2: conocer el dataset
        self._paso3_tipos_variable()                            # ejecuta el
punto 3: tipo de variable
        self._paso4_calidad_datos()                             # ejecuta el
punto 4: calidad de datos
        self._paso5_valores_faltantes()                         # ejecuta el
punto 5: valores faltantes
        self._paso6_outliers()                                  # ejecuta el
punto 6: outliers (IQR + boxplots)
        self._paso7_estadistica_descriptiva()                   # ejecuta el
punto 7: estadística descriptiva
        self._paso8_target()                                    # ejecuta el
punto 8: target / balance de clases
        self._paso9_analisis_univariado()                       # ejecuta el
punto 9: análisis univariado
        self._paso10_bivariado_multivariado()                   # ejecuta el
punto 10: bivariado/multivariado + VIF
        print("=" * 72)                                         # línea separa-
dora inferior
        print(f" DIAGNÓSTICO COMPLETADO · {len(self.hallazgos)} observaciones
registradas ".center(72))  # total de observaciones
        print("=" * 72)                                         # línea separa-
dora
        print("Para la interpretación final ejecuta: eda.obtener_hallazgos()")
# indica cómo recuperar la bitácora

    def obtener_hallazgos(self):
        """Devuelve la bitácora completa de observaciones como DataFrame, insu-
mo para la sección de interpretación."""
        if not self.hallazgos:                                  # caso sin ob-
servaciones registradas
            return pd.DataFrame(columns=['punto', 'tipo', 'detalle'])  # de-
vuelve la tabla vacía con las mismas columnas
        return pd.DataFrame(self.hallazgos)                     # convierte la
lista de dictados en un DataFrame tabular
```